SIMATS ENGINEERING

CO3 - ASSESSMENT TOOL 1
Case Study

COURSE NAME: DEEP LEARNING **COURSE CODE: MLA0408**

COURSE OUTCOME COVERED

CO3: Analyze and apply convolutional neural network architectures, including convolutional layers, filters, parameter sharing, regularization, AlexNet and ResNet, to develop solutions for image classification and real-world computer vision applications. (L4)

CO Weightage: 25%

Assessment Tool Weightage: 25%

QUESTIONS: **Total Marks: 25**

Case Study 1: CNN for Automated Medical Image Classification

A healthcare organization wants to develop a deep learning system to automatically classify chest X-ray images into **normal** and **disease-affected** categories. The dataset contains thousands of X-ray images with variations in size, brightness and image quality. A CNN model is proposed because it can automatically learn important visual features such as edges, textures, and abnormal regions without manually designing features.

Question:

1. Design and explain a suitable **CNN architecture** for this medical image classification system.
2. Describe the **motivation for using CNNs** and explain the role of **convolutional layers, filters, pooling layers, and fully connected layers** in the proposed architecture.
3. Explain how **parameter sharing** reduces the number of trainable parameters compared with a fully connected network.
4. Discuss suitable **regularization techniques such as dropout, data augmentation and batch normalization** to reduce overfitting.
5. Finally, explain whether **AlexNet or ResNet** would be more appropriate for this application and justify your choice based on feature learning, network depth, computational requirements and classification performance.

Case Study 2: CNN-Based Autonomous Vehicle Object Recognition

An autonomous vehicle company is developing a vision system that must identify **cars, pedestrians, traffic signs, bicycles, and road obstacles** from images captured by cameras mounted on the vehicle. The system must recognize objects under different lighting conditions, viewing angles, distances and road environments. The development team is considering **AlexNet and ResNet** architectures for building the CNN-based recognition system.

Question:

1. Analyze how a **CNN architecture** can be designed for this autonomous vehicle application.
2. Explain the purpose of the **input, convolution, activation, pooling, and fully connected/output layers** and describe how convolutional **filters progressively learn low-level and high-level features**.
3. Explain the importance of **parameter sharing** in reducing computational complexity and improving translation-related feature detection.
4. Discuss the possible problem of **overfitting** and recommend suitable regularization methods.
5. Compare **AlexNet and ResNet** in terms of architecture, depth, parameter efficiency, training difficulty, vanishing-gradient problems and feature-learning capability.
6. Finally, recommend the most suitable architecture for the autonomous vehicle system and justify your decision based on **accuracy, computational cost and real-time application requirements**.

Code of Conduct:

I T.Chaitra(192425217)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:T.Chaitra

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding ; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

CASE STUDY 1: CNN FOR AUTOMATED MEDICAL IMAGE CLASSIFICATION

1. Suitable CNN Architecture for Medical Image Classification

A suitable CNN architecture for classifying chest X-ray images into **Normal** and **Disease-Affected** categories can be designed as follows:

Input Image → Preprocessing → Convolution → ReLU → Pooling → Convolution → ReLU → Pooling → Convolution → ReLU → Pooling → Fully Connected → Dropout → Output

Proposed Architecture

Layer	Function
Input Layer	Accepts resized chest X-ray image, e.g., 224 × 224 × 1
Conv Layer 1	Detects basic edges and textures using 32 filters
ReLU	Introduces non-linearity
Max Pooling	Reduces spatial dimensions
Conv Layer 2	Learns intermediate patterns using 64 filters
ReLU	Adds non-linearity
Max Pooling	Reduces feature-map size
Conv Layer 3	Learns complex structures using 128 filters
ReLU	Activates important features
Max Pooling	Further reduces dimensions
Flatten	Converts feature maps into a vector
Fully Connected	Combines learned features for classification
Dropout	Reduces overfitting
Output Layer	Uses sigmoid for Normal/Disease classification

The X-ray images are first resized to a fixed size and normalized. The convolutional layers then automatically extract important visual features. Finally, the fully connected layer performs classification into **Normal** or **Disease-Affected**.

2. Motivation for Using CNNs and Role of Different Layers

Motivation for CNN

CNNs are highly suitable for medical image classification because they can automatically learn useful features directly from images. Traditional machine-learning methods require manually designed features, whereas CNNs learn features such as **edges, textures, shapes, patterns, and abnormal regions** automatically.

CNNs are also effective because they preserve the spatial relationship between pixels and can recognize important patterns even when their position changes within the image.

Convolutional Layer

The convolutional layer applies small filters or kernels to the input image.

For example:

- Early layers → edges and simple textures
- Middle layers → shapes and tissue patterns
- Deeper layers → complex abnormalities and disease-related structures

A convolution operation can be represented as:

Feature Map = Input Image * Filter + Bias

Filters

Filters are small matrices that slide over the image and detect specific patterns.

For example, one filter may detect horizontal edges while another may detect vertical edges or texture patterns.

As training progresses, the CNN automatically learns the most useful filter values.

Pooling Layer

Pooling reduces the spatial dimensions of feature maps while retaining important information.

Max pooling is commonly used because it selects the maximum value from a small region.

Advantages:

- Reduces computation
- Reduces memory requirements
- Helps provide some translation invariance
- Retains important features

Fully Connected Layer

The fully connected layer receives the extracted features and combines them to make the final classification decision.

For binary classification:

Output = Sigmoid(Fully Connected Features)

The output can represent the probability that the X-ray belongs to the disease-affected class.

3. Parameter Sharing

Parameter sharing is one of the major advantages of CNNs.

In a fully connected network, every neuron may need a separate weight for every input pixel. For a **224 × 224** grayscale image:

Number of pixels = 224 × 224 = 50,176

Connecting these pixels to just 100 neurons would require:

50,176 × 100 = 5,017,600 weights

This creates a very large number of parameters.

In a CNN, a **3 × 3 filter** contains only:

3 × 3 = 9 weights + 1 bias

The same filter is applied across the entire image. Therefore, the same parameters are reused at different spatial locations.

This is called **parameter sharing**.

Benefits

- Greatly reduces the number of trainable parameters
- Reduces memory requirements
- Reduces computational complexity
- Helps detect the same feature at different locations
- Improves learning efficiency

4. Regularization Techniques

Medical image datasets may contain thousands of images, but the model can still overfit because medical images can contain variations in equipment, brightness, patient positioning and image quality.

a) Dropout

Dropout randomly deactivates a percentage of neurons during training.

For example, a dropout rate of **0.5** means approximately 50% of selected neurons are temporarily ignored during each training iteration.

Benefits:

- Prevents dependence on particular neurons
- Improves generalization
- Reduces overfitting

b) Data Augmentation

Data augmentation creates modified versions of existing X-ray images.

Possible transformations include:

- Small rotations
- Horizontal shifts
- Zooming
- Brightness adjustment
- Small translations

This increases the effective size and diversity of the training dataset.

Note: Augmentations should be medically appropriate and should not create unrealistic X-ray images.

c) Batch Normalization

Batch normalization normalizes intermediate activations during training.

It helps:

- Stabilize training
- Speed up convergence
- Allow more reliable gradient propagation
- Provide some regularization
- Reduce sensitivity to initialization

Other Techniques

Additional methods include:

- Early stopping
- L2 regularization
- Transfer learning
- Learning-rate scheduling

5. AlexNet vs ResNet

Feature	AlexNet	ResNet
Architecture	Relatively simple CNN	Deep CNN with residual connections
Depth	8 learned layers	Available in many depths such as ResNet-18, 34, 50, 101
Feature Learning	Good	Excellent
Training Deep Networks	More difficult	Easier
Vanishing Gradient	More susceptible	Reduced using skip connections
Parameters	Relatively high for its age	Depends on variant; efficient variants available
Accuracy	Good	Generally higher
Computational Cost	Lower for small versions	Higher for deeper versions
Transfer Learning	Available	Very strong
Medical Image Suitability	Suitable	Highly suitable

Recommended Architecture: ResNet

ResNet is more appropriate for this application, particularly a moderate-sized model such as **ResNet-18 or ResNet-34**, because medical images can contain subtle and complex features.

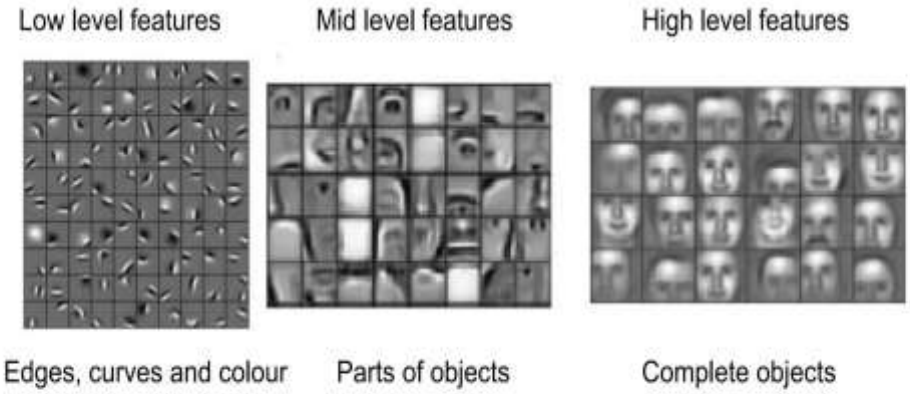
ResNet uses **skip connections**, which allow information and gradients to pass directly through deeper layers. This helps solve the vanishing-gradient problem and makes deep feature learning easier.

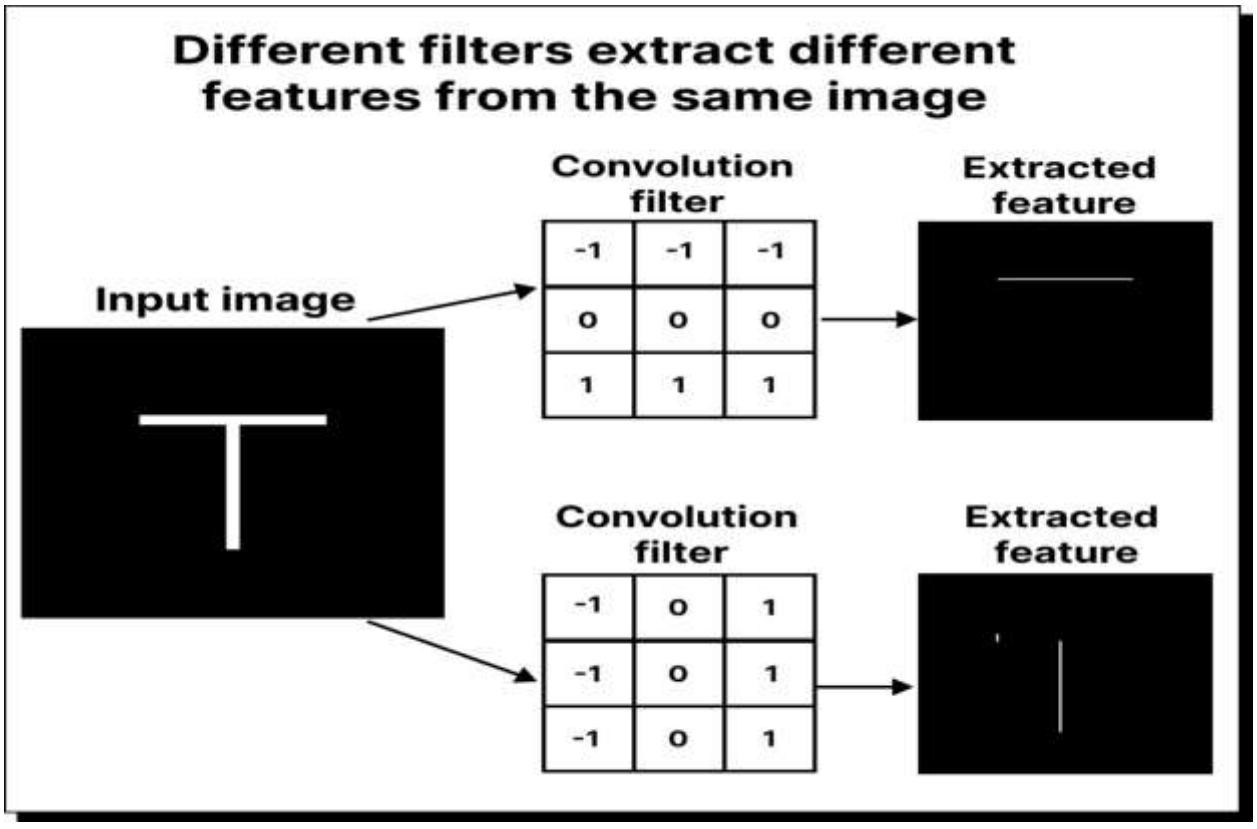
ResNet can learn hierarchical features ranging from simple edges to complex disease-related patterns.

For a medical X-ray system, a pretrained ResNet can also be fine-tuned on the X-ray dataset. This can provide strong performance without training a very deep network entirely from scratch.

Conclusion: ResNet is preferred because it provides stronger feature extraction, better deep-network training and generally better classification performance, while smaller variants such as ResNet-18 can still provide reasonable computational efficiency.

Stages of feature extraction by CNN





CASE STUDY 2: CNN-BASED AUTONOMOUS VEHICLE OBJECT RECOGNITION

1. CNN Architecture for Autonomous Vehicle Object Recognition

An autonomous vehicle needs to recognize multiple objects such as:

- Cars
- Pedestrians
- Bicycles
- Traffic signs
- Road obstacles

A suitable CNN architecture can be designed as:

Camera Image → Preprocessing → Convolution → ReLU → Pooling → Convolution → ReLU → Pooling → Deep Convolution Layers → Fully Connected/Detection Head → Output

For a classification system, the output layer can provide probabilities for different object classes.

For example:

Input → Conv(32) → ReLU → Max Pool → Conv(64) → ReLU → Max Pool → Conv(128) → ReLU → Conv(256) → Global Average Pooling → Fully Connected → Softmax

For a practical autonomous vehicle system, an object-detection architecture would generally be preferred over simple image classification because the vehicle needs to know **what the object is and where it is located**.

2. Purpose of CNN Layers and Progressive Feature Learning

Input Layer

The input layer receives images from vehicle-mounted cameras.

The image is resized and normalized before being provided to the CNN.

Example:

Input = 224 × 224 × 3 RGB image

Convolutional Layer

The convolutional layer applies filters to extract visual features.

The filters slide over different regions of the image and generate feature maps.

Activation Layer

A commonly used activation function is **ReLU**:

ReLU(x) = max(0, x)

It introduces non-linearity and allows the CNN to learn complex patterns.

Pooling Layer

Pooling reduces the spatial size of feature maps.

For example, max pooling can reduce a **224 × 224** feature map to a smaller representation.

This reduces:

- Computation
- Memory usage
- Number of parameters

Fully Connected/Output Layer

The extracted features are combined and passed to the output layer.
For multi-class classification, **Softmax** can be used:
P(class i) = e^zi / Σe^zj
The class with the highest probability is selected.
Progressive Feature Learning
CNN filters learn features hierarchically.
Early layers:

- Edges
- Lines
- Corners
- Simple colors/textures

Middle layers:

- Wheels
- Windows
- Human body parts
- Traffic-sign shapes

Deep layers:

- Cars
- Pedestrians
- Bicycles
- Traffic signs
- Complex road obstacles

Therefore, deeper layers combine simpler features into increasingly meaningful representations.

3. Importance of Parameter Sharing
In CNNs, the same filter weights are reused across different image locations.
For example, a filter trained to detect an edge can detect that edge whether it appears on the **left, center or right** side of the image.
This is known as **parameter sharing**.
Advantages

1. **Reduces number of parameters**
2. **Reduces computational and memory requirements**
3. **Makes training faster**
4. **Allows detection of the same feature at different locations**
5. **Improves translation-related feature detection**

For example, a pedestrian may appear at different positions in a camera image. Because the CNN shares convolutional filters, the same learned pedestrian-related features can be detected at multiple locations.
This property is particularly important in autonomous vehicles because object positions continuously change as the vehicle moves.

4. Overfitting and Regularization
Overfitting occurs when the CNN performs very well on training images but poorly on new road scenes.

This can happen because the model memorizes specific training images instead of learning general object features.

Suitable Regularization Methods

1. Data Augmentation

Generate variations of training images using:

- Rotation
- Cropping
- Scaling
- Brightness changes
- Contrast changes
- Horizontal flipping where appropriate

This helps the model handle different road and lighting conditions.

2. Dropout

Randomly removes neurons during training and prevents the network from becoming overly dependent on specific neurons.

3. Batch Normalization

Normalizes activations and helps stabilize and speed up training while providing some regularization.

4. Weight Decay/L2 Regularization

Penalizes excessively large weights and encourages simpler models.

5. Early Stopping

Training is stopped when validation performance stops improving.

6. Transfer Learning

A pretrained CNN can be fine-tuned using an autonomous-driving dataset, reducing training requirements and improving generalization.

5. AlexNet vs ResNet

Feature	**AlexNet**	**ResNet**
Basic Design	Conventional CNN	CNN with residual/skip connections
Depth	8 learned layers	18, 34, 50, 101, 152+ variants
Feature Learning	Good	Very strong
Training Difficulty	Easier for shallow network	Designed to make very deep networks trainable
Vanishing Gradient	More susceptible	Greatly reduced
Skip Connections	No	Yes
Parameter Efficiency	Less efficient by modern standards	Stronger efficiency/accuracy trade-offs depending on variant
Accuracy	Lower on modern benchmarks	Generally higher

Feature	AlexNet	ResNet
Computational Cost	Lower	Depends on model size
Modern Applications	Mostly historical/baseline	Widely used as backbone/feature extractor

Vanishing-Gradient Problem

In very deep networks, gradients may become extremely small as they are propagated backward through many layers. This makes training difficult.

ResNet addresses this using **skip connections**.

A residual block can be represented as:

Output = F(x) + x

where:

- **x** = original input
- **F(x)** = learned transformation
- **F(x) + x** = residual output

The direct connection allows gradients to flow more easily through the network.

Feature Learning

AlexNet can learn useful visual features, but ResNet's deeper architecture allows it to learn more complex hierarchical features.

For autonomous vehicles, this is useful because the system must distinguish objects under:

- Different distances
- Different viewpoints
- Shadows
- Bright sunlight
- Night conditions
- Partial occlusion
- Complex backgrounds

6. Recommended Architecture for Autonomous Vehicles

Recommended: ResNet

ResNet is generally the better choice for the autonomous vehicle application because it provides stronger feature-learning capability and better performance on complex visual recognition tasks.

A smaller variant such as **ResNet-18 or ResNet-34** can be selected when computational resources and real-time latency are important.

Reasons

1. Higher Accuracy

ResNet's deeper feature extraction can identify complex objects and distinguish visually similar categories.

2. Better Feature Learning

It learns hierarchical features from simple edges to complex object structures.

3. Reduced Vanishing-Gradient Problem

Skip connections make deep networks easier to train.

4. Better Generalization

With appropriate training and regularization, ResNet can generalize well to different road environments.

5. Real-Time Considerations

Instead of choosing a very deep ResNet such as ResNet-152, a lightweight variant such as **ResNet-18** can provide a better balance between accuracy and computational cost.

Final Recommendation

For an autonomous vehicle, **ResNet-18/34 is preferable to AlexNet** when the system requires a balance between **accuracy, feature-learning capability and real-time performance**. However, for actual deployment, the selected model should be benchmarked on the target hardware for FPS, latency, memory usage and detection accuracy.

Overall:

AlexNet → simpler and computationally lighter but older and less powerful. ResNet → deeper, stronger feature learning, better gradient flow and generally better accuracy.

Therefore, **ResNet is the recommended architecture for the autonomous vehicle system.**